PRODUCT REQUIREMENTS DOCUMENT

Face Recognition Attendance System

Powered by InsightFace (ArcFace) — High-Accuracy Biometric Attendance

Document Version	v1.0 — Initial Release	**Date**	March 2026
Status	**Draft — Ready for Development**	**Target Audience**	Developers / GitHub Copilot
Core Engine	**InsightFace — ArcFace (buffalo_l)**	**Similarity Method**	Cosine Similarity (threshold ≥ 0.65)
Language	Python 3.10+	**Storage**	Redis + Firebase Firestore

EXECUTIVE SUMMARY

This PRD defines the complete development specification for a real-time, offline-capable Face Recognition Attendance System built on InsightFace (ArcFace). The system automatically identifies registered individuals via camera, marks their attendance with a timestamp, and prevents fraud through multi-layer liveness detection. It is designed for schools, universities, and corporate offices handling 10 to 5,000+ registered individuals.

Target accuracy: 99%+ on registered faces | Recognition latency: <1 second | Anti-spoofing: 3-layer defense

Table of Contents

1 Purpose & Scope 4
1.1 Purpose 4
1.2 Scope 4
1.3 Target Users 4
2 Technology Stack 4
3 System Architecture 5
3.1 High-Level Architecture 5
3.2 Data Flow 5
3.3 Directory Structure 6
4 Face Engine — InsightFace ArcFace Specification 6
4.1 Model Configuration 6
4.2 Embedding Extraction 7
5 Face Registration Requirements 8
5.1 Registration Workflow 8
5.2 Registration API Endpoint 8
6 Similarity Matching & Threshold Configuration 8
6.1 Cosine Similarity Implementation 9
6.2 Threshold Reference Table 9
7 Liveness Detection & Anti-Spoofing 9
7.1 Overview 9
7.2 Layer 1 — Eye Blink Detection (EAR) 10
7.3 Layer 2 — Texture Analysis (LBP) 10
7.4 Layer 3 — Micro-Movement Detection 10
7.5 Liveness Gate 10
8 Database Schema 10
8.1 Firebase Firestore Collections 10
8.2 Firestore Composite Indexes Required 11
8.3 Firebase Admin SDK Initialization 11
8.4 Redis Key Schema 12
9 REST API Specification 12
9.1 Base Configuration 12
9.2 Endpoint Reference 12
10 Functional Requirements 13
10.1 Core Recognition Requirements 13
10.2 Liveness & Security Requirements 14
11 Non-Functional Requirements 14
12 Testing Plan 15

12.1 Test Categories 15
12.2 Threshold Calibration Procedure 16
13 Deployment & Environment 16
13.1 Docker Compose Configuration 16
13.2 Environment Variables (.env.example) 16
13.3 Hardware Requirements 17
14 Implementation Phases & Milestones 17
15 Acceptance Criteria Summary 18

1 Purpose & Scope

1.1 Purpose

This document provides a complete, implementation-ready specification for the Face Recognition Attendance System (FRAS). It defines all functional and non-functional requirements, data models, API contracts, and testing criteria required for a developer or AI coding assistant (such as GitHub Copilot) to build the system end-to-end without ambiguity.

1.2 Scope

The system SHALL:

- Register new individuals by capturing and encoding their face into a 512-dimensional ArcFace embedding
- Store embeddings in Redis (real-time cache) and attendance records in Firebase Firestore (persistent cloud store)
- Recognize registered faces in real-time via a connected camera and mark attendance automatically
- Enforce liveness detection to prevent spoofing via printed photos, screens, or masks
- Expose a REST API for frontend integration and admin operations
- Provide a Streamlit-based management dashboard for registration, reporting, and administration

The system SHALL NOT:

- Use cloud-based face recognition APIs (all processing must be local/on-premise)
- Store raw face images in any database after embedding extraction (privacy-by-design)
- Rely on the Gemini API, Google Vision API, or any external ML inference endpoint

1.3 Target Users

User Type	Description	Primary Interactions
Administrator	Manages the system, registers users, views reports	Registration, configuration, export, audit logs
Attendee	Student, employee, or member being tracked	Stands in front of camera; no app interaction
Teacher / Supervisor	Reviews attendance reports for their group	Dashboard read access, export to CSV/Excel

2 Technology Stack

All components below are mandatory unless marked OPTIONAL. The stack is chosen for maximum accuracy, offline operation, and open-source licensing.

Layer	Technology	Version	Purpose
Face Recognition	**InsightFace (ArcFace)**	buffalo_l model	512-dim embedding extraction

Layer	Technology	Version	Purpose
Face Detection	**InsightFace RetinaFace**	Built into InsightFace	Bounding box detection
Liveness Detection	**MediaPipe Face Mesh**	v0.10+	Eye blink / EAR-based liveness
Similarity Matching	**NumPy Cosine Similarity**	1.24+	Embedding vector comparison
Real-Time Cache	**Redis**	7.x	Embedding lookup, session locking
Persistent Database	**Firebase Firestore**	Admin SDK v6+	Attendance records, user registry (NoSQL cloud)
Firebase Auth	**Firebase Authentication**	Admin SDK v6+	User identity & JWT token management
API Backend	**FastAPI**	0.110+	REST API with OpenAPI docs
Video Capture	**OpenCV (cv2)**	4.9+	Camera frame capture
Dashboard	**Streamlit**	1.35+	Admin UI (registration & reports)
Task Queue (OPT)	**Celery + Redis**	Latest	Async attendance logging
Containerization	**Docker + Docker Compose**	Latest	Reproducible deployment

3 System Architecture

3.1 High-Level Architecture

The system follows a layered pipeline architecture consisting of five core modules that interact through clearly defined interfaces.

Each step is independently testable and loosely coupled via well-defined Python class interfaces.

3.2 Data Flow

1. Camera captures a frame at 15–30 FPS via OpenCV VideoCapture
2. RetinaFace (via InsightFace) detects face bounding boxes in the frame
3. Liveness engine (MediaPipe EAR) confirms the face is live — rejects photos and videos
4. ArcFace model generates a 512-dimensional embedding for the detected face
5. Cosine similarity compared against all embeddings in Redis; best match selected

6. If similarity >= threshold (0.65), attendance is marked in Firebase Firestore with timestamp
7. Result (name, confidence, timestamp) is displayed on the camera overlay

3.3 Directory Structure

The project SHALL follow this directory structure to ensure clear code organization:

```
fras/
├── app/
│   ├── main.py              # FastAPI application entry point
│   ├── config.py            # Environment variables and thresholds
│   ├── models/
│   │   ├── user.py          # Firestore User document model
│   │   └── attendance.py    # Firestore Attendance document model
│   ├── services/
│   │   ├── face_engine.py   # InsightFace wrapper (detect + embed)
│   │   ├── liveness.py      # MediaPipe liveness detection
│   │   ├── matcher.py       # Cosine similarity matching
│   │   └── attendance.py    # Attendance logging service
│   ├── api/
│   │   ├── routes/
│   │   │   ├── users.py     # User registration endpoints
│   │   │   ├── attendance.py # Attendance query endpoints
│   │   │   └── camera.py    # Camera stream endpoints
│   │   └── schemas.py       # Pydantic request/response models
│   └── db/
│       ├── firebase_client.py # Firebase Admin SDK init + Firestore
│       └── redis_client.py  # Redis connection + embedding cache
├── dashboard/
│   └── app.py              # Streamlit admin dashboard
├── camera/
│   └── capture.py          # OpenCV camera loop
├── tests/
│   ├── test_face_engine.py
│   ├── test_liveness.py
│   ├── test_matcher.py
│   └── test_api.py
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
└── .env.example
```

4 Face Engine — InsightFace ArcFace Specification

4.1 Model Configuration

The FaceEngine class MUST use the following InsightFace configuration exactly:

```
import insightface
from insightface.app import FaceAnalysis

class FaceEngine:
    MODEL_NAME  = "buffalo_l"      # ArcFace — highest accuracy model
```

```
    CTX_ID      = 0                 # 0 = GPU, -1 = CPU
    DET_SIZE    = (640, 640)        # Detection resolution — do NOT lower
    DET_THRESH  = 0.5              # Minimum detection confidence

    def __init__(self):
        self.app = FaceAnalysis(name=self.MODEL_NAME, providers=['CUDAExecutionProvider',
'CPUExecutionProvider'])
        self.app.prepare(ctx_id=self.CTX_ID, det_size=self.DET_SIZE)
```

Parameter	Value	Rationale
Model	**buffalo_l**	Largest ArcFace model — best accuracy/speed trade-off
Embedding Dimensions	**512**	Standard ArcFace output size
Detection Input Size	**640 × 640 px**	Higher resolution = better small-face detection
Inference Provider	**CUDA → CPU fallback**	GPU if available, auto-falls back to CPU
Normalization	**L2-norm on embeddings**	Required for cosine similarity to work correctly
Max Faces Per Frame	**10**	Process up to 10 simultaneous faces per frame

4.2 Embedding Extraction

The get_embedding() method MUST follow this exact implementation contract:

```
import numpy as np

def get_embedding(self, frame: np.ndarray) -> tuple[np.ndarray | None, dict | None]:
    """
    Returns (embedding, face_metadata) or (None, None) if no valid face detected.
    """
    faces = self.app.get(frame)
    if not faces:
        return None, None

    # Select the largest face (closest to camera)
    face = max(faces, key=lambda f: (f.bbox[2]-f.bbox[0]) * (f.bbox[3]-f.bbox[1]))

    # L2-normalize the embedding for cosine similarity
    embedding = face.embedding
    embedding = embedding / np.linalg.norm(embedding)

    meta = {
        "bbox": face.bbox.tolist(),
        "det_score": float(face.det_score),
        "age": int(face.age) if hasattr(face, "age") else None,
        "gender": face.gender if hasattr(face, "gender") else None,
    }
    return embedding, meta
```

5 Face Registration Requirements

5.1 Registration Workflow

Registration is the most critical step. Poor registration directly leads to wrong-person matching. The system MUST enforce the following multi-step protocol:

Step	Action	Technical Requirement
1	**Collect 10 frames**	Capture exactly 10 frames over 5 seconds (2 fps). Reject any frame where det_score < 0.7.
2	**Quality validation**	Each frame MUST pass: single face only, face width > 120px, no heavy blur (Laplacian variance > 100).
3	**Pose diversity check**	At least 3 different head poses must be represented (frontal, left-tilt, right-tilt) using yaw angle from InsightFace.
4	**Embedding extraction**	Run ArcFace on all valid frames. L2-normalize each resulting 512-dim embedding.
5	**Mean embedding**	Average all valid embeddings into one mean embedding vector. L2-normalize the final mean.
6	**Duplicate check**	Before saving, compare against all existing embeddings. If cosine similarity >= 0.75 with any existing user, reject and alert.
7	**Store embedding**	Save embedding to Redis (key: user:{id}:embedding) AND user profile to Firestore /users/{id}. Raw images are DISCARDED after step 4.

5.2 Registration API Endpoint

```
POST /api/v1/users/register

Request (multipart/form-data):
  name         : str   — Full name of the person
  employee_id  : str   — Unique ID (student/employee number)
  department   : str   — Department or class name
  role         : str   — 'student' | 'employee' | 'teacher'
  frames[]     : File  — 10 image files (JPEG/PNG)

Response 200 OK:
  { user_id, name, employee_id, embedding_dimensions: 512, registered_at }

Response 409 Conflict (duplicate face):
  { error: "DUPLICATE_FACE", matched_user_id, similarity_score }

Response 422 Unprocessable:
  { error: "POOR_QUALITY", failed_frames: [2, 5, 7], reason: "..." }
```

6 Similarity Matching & Threshold Configuration

6.1 Cosine Similarity Implementation

```
import numpy as np
from redis import Redis

class FaceMatcher:
    MATCH_THRESHOLD   = 0.65  # Minimum cosine similarity to accept a match
    UNKNOWN_THRESHOLD = 0.50  # Below this = completely unknown person

    def find_match(self, query_embedding: np.ndarray) -> dict:
        """Compare query embedding against all stored embeddings in Redis."""
        best_match = None
        best_score = -1.0

        for key in self.redis.scan_iter('user:*:embedding'):
            stored = np.frombuffer(self.redis.get(key), dtype=np.float32)
            score  = float(np.dot(query_embedding, stored))  # both L2-normed

            if score > best_score:
                best_score = score
                best_match = key.decode().split(':')[1]  # user_id

        if best_score >= self.MATCH_THRESHOLD:
            return {"matched": True,  "user_id": best_match, "confidence": best_score}
        elif best_score >= self.UNKNOWN_THRESHOLD:
            return {"matched": False, "user_id": None, "confidence": best_score, "status":
"LOW_CONFIDENCE"}
        else:
            return {"matched": False, "user_id": None, "confidence": best_score, "status":
"UNKNOWN"}
```

6.2 Threshold Reference Table

Similarity Score	Interpretation	System Action	Risk Level
≥ 0.85	Very high confidence match	Mark attendance immediately	**Very Low**
0.65 – 0.84	Standard match	Mark attendance	**Low**
0.50 – 0.64	Low confidence	Flag for review, no auto-mark	**Medium**
< 0.50	Unknown person	Reject — log as unknown attempt	**High**

7 Liveness Detection & Anti-Spoofing

7.1 Overview

Liveness detection is the primary defense against wrong-person marking. The system MUST implement all three layers below. Attendance is only marked when ALL layers pass.

7.2 Layer 1 — Eye Blink Detection (EAR)

Eye Aspect Ratio (EAR) measures eye openness. A blink is confirmed when EAR drops below 0.25 and returns above it. This cannot be faked with a static photo or video.

```
import mediapipe as mp
from scipy.spatial import distance

EAR_THRESHOLD  = 0.25   # Below this = eye closed
BLINK_FRAMES   = 3      # Eye must be closed for at least 3 consecutive frames
BLINK_REQUIRED = 2      # Require 2 blinks before accepting

def compute_ear(eye_landmarks) -> float:
    A = distance.euclidean(eye_landmarks[1], eye_landmarks[5])
    B = distance.euclidean(eye_landmarks[2], eye_landmarks[4])
    C = distance.euclidean(eye_landmarks[0], eye_landmarks[3])
    return (A + B) / (2.0 * C)
```

7.3 Layer 2 — Texture Analysis (LBP)

Local Binary Pattern (LBP) analysis detects print artifacts. Real skin has micro-texture that printed photos lack. The system SHALL compute LBP variance on the face ROI — reject if variance falls below 8.0 (indicates flat printed surface).

7.4 Layer 3 — Micro-Movement Detection

Track facial landmark positions across 10 consecutive frames. A real person shows subtle involuntary micro-movements (breathing, natural head drift). A static image or video replay shows artificially smooth or frozen landmark positions. REJECT if mean inter-frame landmark displacement < 0.5 pixels over 10 frames.

7.5 Liveness Gate

```
class LivenessGate:
    def evaluate(self, session: LivenessSession) -> LivenessResult:
        results = {
            "blink_passed"    : session.blink_count >= BLINK_REQUIRED,
            "texture_passed"  : session.lbp_variance >= 8.0,
            "movement_passed" : session.mean_displacement >= 0.5,
        }
        all_passed = all(results.values())
        return LivenessResult(is_live=all_passed, details=results)

# Attendance is ONLY triggered when is_live == True
```

8 Database Schema

8.1 Firebase Firestore Collections

Firestore uses a document-collection model. The system uses three top-level collections. All document IDs are auto-generated by Firestore unless noted.

Collection: /users/{userId}

```
// Document ID: auto-generated Firestore ID (also stored in Firebase Auth uid)
{
  "userId"        : "string  — Firestore doc ID (matches Firebase Auth UID)",
```

```
    "employeeId"    : "string  — Unique staff/student number  [INDEXED]",
    "name"          : "string  — Full name",
    "department"    : "string  — Department or class",
    "role"          : "string  — Enum: student | employee | teacher | admin",
    "registeredAt"  : "Timestamp — Firestore server timestamp",
    "isActive"      : "boolean — Soft-delete flag (default: true)",
    "embeddingDim"  : "number  — Always 512 for ArcFace buffalo_l",
    "embeddingRef"  : "string  — Redis key: user:{userId}:embedding  (NOT stored here)"
  }
```

Collection: /attendance/{recordId}

```
  // Composite key enforced via Redis SET prevents duplicates in the same session
  {
    "recordId"      : "string  — Auto-generated Firestore doc ID",
    "userId"        : "string  — Reference to /users/{userId}  [INDEXED]",
    "sessionId"     : "string  — Format: YYYY-MM-DD__{shiftOrClass}  [INDEXED]",
    "markedAt"      : "Timestamp — Firestore server timestamp  [INDEXED]",
    "confidence"    : "number  — Cosine similarity score (0.0 - 1.0)",
    "livenessScore" : "number  — Aggregated liveness confidence (0.0 - 1.0)",
    "cameraId"      : "string  — Camera identifier",
    "status"        : "string  — Enum: present | late | absent"
  }
```

Collection: /spoofAttempts/{attemptId}

```
  {
    "attemptId"     : "string  — Auto-generated Firestore doc ID",
    "attemptedAt"   : "Timestamp  [INDEXED]",
    "failureReason" : "string  — Enum: NO_BLINK | FLAT_TEXTURE | NO_MOVEMENT |
LOW_CONFIDENCE",
    "cameraId"      : "string",
    "snapshotHash"  : "string  — SHA256 of frame (raw image is NEVER stored)"
  }
```

8.2 Firestore Composite Indexes Required

Collection	Fields Indexed	Query Use Case
/attendance	userId ASC + markedAt DESC	Attendance history for a single user
/attendance	sessionId ASC + markedAt ASC	All attendees in a specific session
/attendance	userId ASC + sessionId ASC	Deduplication check (unique per session)
/spoofAttempts	cameraId ASC + attemptedAt DESC	Spoof attempts per camera

8.3 Firebase Admin SDK Initialization

```
  # app/db/firebase_client.py
  import firebase_admin
```

```
from firebase_admin import credentials, firestore
import os

_db = None

def get_firestore() -> firestore.Client:
    global _db
    if _db is None:
        cred = credentials.Certificate(os.environ['FIREBASE_SERVICE_ACCOUNT_JSON'])
        firebase_admin.initialize_app(cred)
        _db = firestore.client()
    return _db

# Usage in services:
# db = get_firestore()
# db.collection('attendance').add({ ...document... })
# db.collection('users').document(user_id).set({ ...document... })
```

8.4 Redis Key Schema

Key Pattern	Data Type	Description
user:{uuid}:embedding	Bytes (float32 array)	512-dim L2-normalized ArcFace embedding
user:{uuid}:meta	Hash	name, employee_id, department, role
session:{date}:marked	Set	Set of user UUIDs already marked today
camera:{id}:lock	String (TTL 2s)	Prevents duplicate processing of same frame
spoof:{camera_id}:count	Integer (TTL 1hr)	Spoof attempt counter per camera per hour

9 REST API Specification

9.1 Base Configuration

Base URL: http://localhost:8000/api/v1 | Auth: Bearer JWT | Format: application/json

9.2 Endpoint Reference

Method	Endpoint	Description	Auth Required
POST	/users/register	Register new user with face images	Admin
GET	/users/{user_id}	Get user profile and embedding info	Admin
DELETE	/users/{user_id}	Remove user and their embedding	Admin

Method	Endpoint	Description	Auth Required
GET	/users/	List all registered users (paginated)	Admin
POST	/attendance/mark	Mark attendance from image frame	Camera / API
GET	/attendance/	Query attendance records with filters	Admin/Teacher
GET	/attendance/user/{user_id}	Get attendance history for one user	Admin/Teacher
GET	/attendance/export	Export attendance as CSV/Excel	Admin/Teacher
GET	/camera/stream	WebSocket: live annotated video feed	Admin
GET	/health	System health check	None

10 Functional Requirements

10.1 Core Recognition Requirements

ID	Requirement	Description	Priority
FR-01	**Real-Time Recognition**	System MUST recognize registered faces in real-time from live camera feed within 1 second per frame.	Must Have
FR-02	**Accuracy Target**	Face recognition accuracy MUST be ≥ 99% True Accept Rate for registered users under normal lighting conditions.	Must Have
FR-03	**False Accept Rate**	False Accept Rate (wrong person marked) MUST be < 0.1%. Enforced by cosine threshold ≥ 0.65.	Must Have
FR-04	**Multi-Face Frame**	System MUST handle frames with up to 10 faces simultaneously, marking attendance for all valid matches.	Must Have
FR-05	**Session Deduplication**	Each user MUST only be marked once per session (date + session_id). Redis SET prevents duplicates.	Must Have
FR-06	**Confidence Display**	Camera overlay MUST show: name, confidence %, bounding box, and liveness status for each detected face.	Must Have
FR-07	**Unknown Handling**	Unknown faces MUST be logged as UNKNOWN_ATTEMPT with frame hash (no image stored) for audit purposes.	Must Have
FR-08	**Batch Registration**	Admin MUST be able to register up to 50 users via CSV + photo zip import.	Should Have
FR-09	**Embedding Update**	Admin MUST be able to re-register a user to update their embedding without changing their attendance history.	Should Have

ID	Requirement	Description	Priority
FR-10	**Offline Operation**	System MUST function without internet connectivity. All inference is local.	Must Have

10.2 Liveness & Security Requirements

ID	Requirement	Description	Priority
SR-01	**Blink Liveness**	System MUST require ≥ 2 blinks (EAR < 0.25 for 3+ frames) before accepting attendance.	Must Have
SR-02	**Print Attack Defense**	System MUST reject printed photo attacks via LBP texture analysis (variance threshold < 8.0 = reject).	Must Have
SR-03	**Screen Attack Defense**	System MUST detect and reject video/screen replay via micro-movement analysis (displacement < 0.5px = reject).	Must Have
SR-04	**Spoof Logging**	ALL failed liveness checks MUST be logged to spoof_attempts table with camera ID and failure reason.	Must Have
SR-05	**Rate Limiting**	Camera endpoint MUST enforce rate limit: max 5 failed liveness attempts per camera per hour before alerting admin.	Should Have
SR-06	**No Raw Image Storage**	Raw face images MUST NEVER be persisted to database or disk after embedding extraction.	Must Have
SR-07	**Admin JWT Auth**	All admin API endpoints MUST require valid JWT token. Tokens expire in 8 hours.	Must Have

11 Non-Functional Requirements

Category	Requirement	Acceptance Criteria
Performance	Recognition latency	End-to-end: detect + embed + match < 1,000ms on GPU; < 3,000ms on CPU-only
Performance	Camera throughput	Process ≥ 15 FPS from a single camera stream without frame drops
Performance	Scalability	Redis embedding lookup < 50ms for up to 10,000 registered users
Performance	API response time	All REST endpoints respond within 200ms (excluding mark attendance)
Reliability	System uptime	99.5% uptime during operating hours; auto-restart via Docker health checks
Reliability	Graceful degradation	If GPU unavailable, system falls back to CPU and logs a warning
Privacy	No PII storage	No raw images, no biometric data except L2-normalized embeddings

Category	Requirement	Acceptance Criteria
Privacy	Data retention	Firestore /attendance documents deleted after 3 years via a scheduled Cloud Function or admin cron job (configurable via ATTENDANCE_RETENTION_DAYS env var)
Security	Embedding encryption	Embeddings stored in Redis MUST be encrypted at rest using AES-256
Usability	Recognition distance	System MUST work reliably at 0.5m to 2.0m camera distance
Usability	Lighting range	System MUST work in 100 to 10,000 lux lighting conditions

12 Testing Plan

12.1 Test Categories

Test Type	Test Cases	Expected Result
Unit Tests	get_embedding() with valid face image	Returns 512-dim L2-normalized numpy array
Unit Tests	compute_ear() with known landmark coords	Returns float between 0.0 and 1.0
Unit Tests	find_match() with identical embedding	Returns similarity = 1.0, matched = True
Unit Tests	find_match() with random embedding	Returns matched = False, status = UNKNOWN
Integration Tests	Register 10 users, verify Redis keys created	10 embedding keys in Redis, 10 documents in Firestore /users
Integration Tests	Mark attendance via API with valid face	Firestore /attendance document created, confidence >= 0.65
Anti-Spoofing Tests	Hold printed photo to camera	Rejected: liveness = False, spoof_attempts log entry created
Anti-Spoofing Tests	Show video of registered user on phone	Rejected: micro-movement check fails
Anti-Spoofing Tests	Register face and mark attendance correctly	Accepted: all 3 liveness layers pass
Threshold Tests	Test 100 registered users, 100 unknown faces	FAR < 0.1%, FRR < 1% at threshold 0.65
Load Tests	10 simultaneous camera streams	All respond within 3s, no crashes
Regression Tests	Re-register user and verify new embedding	Old embedding replaced, attendance history intact

12.2 Threshold Calibration Procedure

8. Register 30 test users with 10 images each
9. Collect 50 positive samples (registered users) and 50 negative samples (unregistered faces)
10. Run all 100 samples through the matcher at thresholds: 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80
11. Plot FAR vs FRR for each threshold value (ROC curve)
12. Select the threshold where FAR < 0.1% AND FRR < 1.0% — this should be approximately 0.65 for buffalo_l
13. Document the final threshold value in config.py as MATCH_THRESHOLD

13 Deployment & Environment

13.1 Docker Compose Configuration

```
  version: '3.9'
  services:
    api:
      build: .
      ports: ['8000:8000']
      volumes: ['./app:/app']
      environment:
        - REDIS_URL=redis://redis:6379
        - FIREBASE_SERVICE_ACCOUNT_JSON=/run/secrets/firebase_sa
        - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}
        - MATCH_THRESHOLD=0.65
        - LIVENESS_REQUIRED=true
      secrets: [firebase_sa]
      depends_on: [redis]
      runtime: nvidia    # Remove if no GPU
      healthcheck:
        test: ['CMD', 'curl', '-f', 'http://localhost:8000/api/v1/health']
        interval: 30s
    redis:
      image: redis:7-alpine
      ports: ['6379:6379']
      command: redis-server --requirepass ${REDIS_PASSWORD} --save 60 1
    dashboard:
      build: ./dashboard
      ports: ['8501:8501']
      environment:
        - FIREBASE_SERVICE_ACCOUNT_JSON=/run/secrets/firebase_sa
        - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}
      secrets: [firebase_sa]
      depends_on: [api]
  secrets:
    firebase_sa:
      file: ./secrets/firebase-service-account.json
  # NOTE: No postgres container — Firestore is a fully managed cloud service by Google
Firebase.
```

13.2 Environment Variables (.env.example)

All configuration is managed via environment variables. The following MUST be defined before running the application:

```
# Firebase
FIREBASE_PROJECT_ID=your-firebase-project-id
FIREBASE_SERVICE_ACCOUNT_JSON=/run/secrets/firebase-service-account.json

# Redis
REDIS_URL=redis://localhost:6379
REDIS_PASSWORD=your_redis_password

# InsightFace
INSIGHTFACE_MODEL=buffalo_l
MATCH_THRESHOLD=0.65
LIVENESS_REQUIRED=true

# Camera
CAMERA_INDEX=0          # 0 = default webcam; or RTSP URL for IP camera
CAMERA_FPS=15

# API
API_HOST=0.0.0.0
API_PORT=8000

# Data Retention
ATTENDANCE_RETENTION_DAYS=1095   # 3 years
```

13.3 Hardware Requirements

Component	Minimum (CPU-only)	Recommended (GPU)
CPU	4-core x86_64	**8-core x86_64 (Intel/AMD)**
RAM	8 GB	**16 GB**
GPU	None (CPU fallback)	**NVIDIA with CUDA 11.8+ (4GB VRAM min)**
Storage	20 GB SSD	**50 GB NVMe SSD**
Camera	1080p USB webcam (30fps)	**1080p IP camera or industrial USB cam**
OS	Ubuntu 22.04 LTS	**Ubuntu 22.04 LTS**

14 Implementation Phases & Milestones

Phase	Title	Deliverables	Acceptance Criteria
1	**Core Engine**	FaceEngine class, ArcFace embedding extraction, L2 normalization, basic cosine matcher	get_embedding() returns 512-dim vector; matcher finds correct user at similarity ≥ 0.65

Phase	Title	Deliverables	Acceptance Criteria
2	**Registration**	Registration API, 10-frame capture, quality validation, Redis + Firestore storage, duplicate detection	Register 10 users; verify embeddings in Redis and documents in Firestore; duplicate check triggers
3	**Liveness**	MediaPipe EAR blink detection, LBP texture analysis, micro-movement detection, LivenessGate class	Photo/screen attacks rejected; real face accepted after 2 blinks
4	**API Layer**	FastAPI app with all endpoints, JWT auth, Pydantic schemas, OpenAPI docs	All 10 endpoints return correct responses; auth works; docs at /docs
5	**Dashboard**	Streamlit admin UI: registration wizard, attendance report, export, camera live feed	Admin can register user and view their attendance history via UI
6	**Testing**	pytest suite for all units, integration tests, anti-spoofing tests, threshold calibration	All tests pass; FAR < 0.1% confirmed at threshold 0.65
7	**Deployment**	Docker Compose config, .env setup, health checks, README	docker compose up starts full system; accessible at localhost:8000 and :8501

15 Acceptance Criteria Summary

The system is considered complete and production-ready when ALL of the following criteria are verified:

#	Acceptance Criterion	Verification Method
1	A registered user is recognized in < 1 second and their attendance is marked automatically	Live camera test
2	An unknown/unregistered person is rejected with status UNKNOWN — no false attendance record created	Live camera test
3	A printed photo of a registered person is rejected by the liveness gate	Anti-spoofing test
4	A video played on a phone of a registered person is rejected	Anti-spoofing test
5	FAR (False Accept Rate) is confirmed < 0.1% via 100-sample threshold calibration test	pytest threshold test
6	Duplicate registration is prevented — same face registered twice triggers DUPLICATE_FACE error	API integration test
7	Same person cannot be marked twice in the same session — Redis deduplication works	Unit + integration test
8	All REST endpoints return correct HTTP status codes and response schemas	pytest API tests

#	Acceptance Criterion	Verification Method
9	Docker Compose starts the complete system with one command: docker compose up	Deployment test
10	Admin dashboard shows real-time attendance and allows CSV export	Manual UI test

END OF DOCUMENT

Face Recognition Attendance System PRD | InsightFace ArcFace | v1.0 | March 2026